

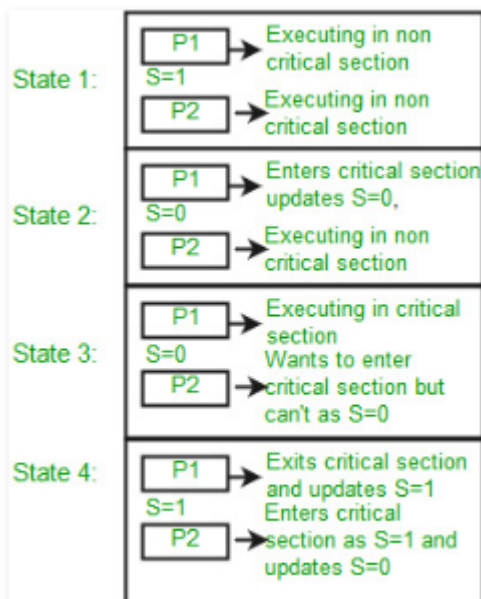
Chap5_Process_Synchronization

Chap5_Process_Synchronization

Critical Section problem

Solving the critical section problem

- Petersen's Software solution
- Synchronization Hardware
 - The Hardware Approach of synchronization can be done through Lock & Unlock technique. Locking part is done in the Entry Section, so that only one process is allowed to enter into the Critical Section, after it completes its execution, the process is moved to the Exit Section, where Unlock Operation is done so that another process in the Lock Section can repeat this process of Execution. This process is designed in such a way that all the three conditions of the Critical Sections are satisfied
- Semaphore
 - Integer variable for which only two (**atomic**) operations are defined, the wait and signal operations.
 - Binary
 - This is also known as mutex lock. It can have only two values – 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problem with multiple processes.



- Counting semaphore
 - Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.
 - The value is initialised to number of such resources available in the system.
 - Whenever the counting semaphore is >0 , a process can enter its critical section, otherwise not.
 - When it is ≤ 0 , the process gives out a signal call.
- Limitations of semaphores
 - One of the biggest limitation of semaphore is priority inversion.
 - Deadlock, suppose a process is trying to wake up another process which is not in sleep state. Therefore a deadlock may block indefinitely. **When multiple processes are blocked, each waiting for a resource that can only be freed by one of the other (blocked) processes.**
 - **starvation**, in which one or more processes gets blocked forever, and never get a chance to take their turn in the critical section. For example, in the semaphores above, we did not specify the algorithms for adding processes to the waiting queue in the semaphore in the wait() call, or selecting one to be removed from the queue in the signal() call. If the method chosen is a FIFO queue, then every process will eventually get their turn, but if a LIFO queue is implemented instead, then the first process to start waiting could starve
 - The operating system has to keep track of all calls to wait and to signal the semaphore.
 - **Priority Inversion**
 - High priority processes are blocked

https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/5_Synchronization.html

- Classic Problems of synchronization
 - The bounded buffer problem
 - The reader's writer problem
 - The dining philosopher problem
- Monitors
 -

The critical Section Problem

- The processes may be changing common variables, updating tables, writing files etc.
This is the **critical Section of the process**

•

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

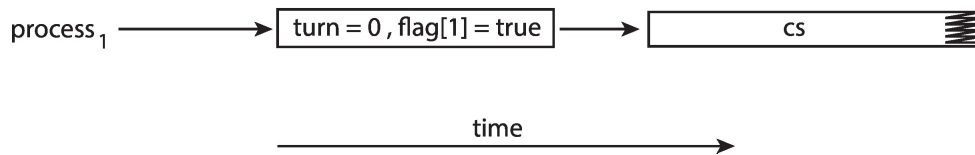
•

- Requirements for solution to Critical Section Problem
 - **Mutual Exclusion**
 - **Progress** - if no process executing in its critical section, and there are process wishing to enter their critical section, then the selection of the **process that will enter the critical section next cannot be postponed indefinitely**
 - **Bounded Waiting** - After a process has requested to enter its critical section, it would have to wait a max bound number of processes before it is surely given a chance

Peterson's Solution

- The two processes share two variables:
 - int turn;
 - boolean flag[2]
 - The variable turn indicates whose turn it is to enter the critical section
 - The flag array is used to indicate if a process is ready to enter the critical section. **Shows the processes which are waiting to enter the critical section.**
 - flag[i] = true implies that process P_i is ready!
 - All 3 conditions satisfied
 - **Solution works for single thread processors, but not for multithreaded processors**
-





- Solution to this multithreaded env pproblem is **MEMORY BARRIER**, an instruciton that forces any change in memory to be propagated to all other processors.
- **Memory models** are the memory gaurantees a computer arch makes to application programs. They are either :
 - Strongly ordered - memory modification of one processor is immediatly visible to all other processes.
 - Weakly ordered - may not be mmediatly visible to all other processors



Memory Barrier Example

- Returning to the example of slides 6.17 - 6.18
- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs


```
while (!flag)
    memory_barrier();
print x
```
- Thread 2 now performs


```
x = 100;
memory_barrier();
flag = true
```
- For Thread 1 we are guaranteed that that the value of `flag` is loaded before the value of `x`.
- For Thread 2 we ensure that the assignment to `x` occurs before the assignment `flag`.

◦

Another Solution - Synchronization hardware

- Uniprocessors - could disable interrupts when a shared varibale is being modified
- But the uniprocessor approach cannot work in a multiprocessor environment. Interrupt disabling could be time consuming. **SO we use sspecial hardware instructions which allow us eeither to test and modify the content of the word or to swap the contents of two words atomically.**
- Different forms of hardware support
 - **Hardware intructions**
 - **Test and Set (executed atomically)**

- ```

boolean test_and_set(boolean *target) {
 boolean rv = *target;
 *target = true;

 return rv;
}

```

**Figure 5.3** The definition of the `test_and_set()` instruction

- ```

do {
    while (test_and_set(&lock))
        ; /* do nothing */

    /* critical section */

    lock = false;

    /* remainder section */
} while (true);

```

Figure 5.4 Mutual-exclusion implementation with `test_and_set()`.

-

▪ Compare and swap

- ```

int compare_and_swap(int *value, int expected, int new_value) {
 int temp = *value;

 if (*value == expected)
 *value = new_value;

 return temp;
}

```

A value of 1 is set and subsequent calls to `compare&swap` do not succeed. A value of 1 indicates it is occupied.

- ```

do {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
} while (true);

```

Figure 5.6 Mutual-exclusion implementation with the `compare_and_swap()` instruction.

-

- Although these algorithms satisfy the mutual-exclusion requirement, they do not satisfy the bounded-waiting requirement.
 - The bounded waiting + mutual exclusion code

- `boolean waiting[n];`
`boolean lock;`
- The above two are all initialized to false. A `lock==true` means lock is already acquired by a process. `waiting[i]==true` means the process is waiting to enter its critical section
- ```
do {
 waiting[i] = true;
 key = true;
 while (waiting[i] && key)
 key = test_and_set(&lock);
 waiting[i] = false;

 /* critical section */

 j = (i + 1) % n;
 while ((j != i) && !waiting[j])
 j = (j + 1) % n;

 if (j == i)
 lock = false;
 else
 waiting[j] = false;

 /* remainder section */
} while (true);
```

**Figure 5.7** Bounded-waiting mutual exclusion with `test_and_set()`.

- Steps in a process's life:
  1. **As `waiting[i]` and `key` both become true**, the process acquires the lock, **`key`→false, `lock`→true**
  2. **`waiting[i]`→false** because the process is now executing and not waiting
  3. Critical Section executes.
  4. Then the waiting array is scanned to **find next waiting process**.
  5. Meanwhile the scan is done, any process waiting to enter its critical section will thus enter its critical section within the  $n-1$  turns.
  6. As a waiting process is found (not same as  $i$ ) then it is allowed to execute (`waiting[j]`→false and `key=true`) throws it out of its loop

## Mutex locks

- The hardware-based solutions is complicated as well as generally inaccessible to application programmers.
- mutex lock to protect critical regions and thus prevent race conditions. That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section.

```

do {
 acquire lock
 critical section
 release lock
 remainder section
} while (true);

```

Solution to the critical-section problem using mutex locks.

- Disad-
  - Requires **busy waiting**
  - While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to acquire().
  - In fact, this type of mutex lock is also called a **spinlock** because the process "spins" while waiting for the lock to become available
- Adv -
  - Spinlocks do have an advantage, however, in that **no context switch is required** when a process must wait on a lock, and a context switch may take considerable time.

## Semaphores

- Behave similarly to mutex, but more sophisticated
- Semaphore S is an integer variable **accessed only thru 2 atomix ops- called wait() and signal().**
- **wait()** → P (Dutch probere "to test"); **signal()** → V()(Dutch verhogen, "to increment")
- 

```

wait(S) {
 while (S <= 0)
 ; // busy wait
 S--;
}

```

of signal() is as follows:

```

signal(S) {
 S++;
}

```

- Two types:
  - Binary semaphores - mutex lock functionality
  - Counting semaphores
    - Initialized to #resources available
    - Decrementd when a process occupies it. (**wait()** )
    - Incremented when a process finishes CS and releases resource.( **signal()** op)

- **Busy waiting** problem's solution

- Rather than looping continuously to check for free resource, the process can block itself.
- The blocked process is now in **waiting queue** associated with the semaphore.
- When this process needs to be restarted, when some other process calls **signal()** op. So it calls the **wakeup** op.
- Now the process is in ready queue.

- To implement semaphores under this definition, we define a semaphore as follows:

```
typedef struct {
 int value;
 struct process *list;
} semaphore;
```

Each semaphore has an integer value and a list of processes *list*. When a process must wait on a semaphore, it is added to the list of processes. A **signal()** operation removes one process from the list of waiting processes and awakens that process.

- ```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

- ```
signal(semaphore *S) {
 S->value++;
 if (S->value <= 0) {
 remove a process P from S->list;
 wakeup(P);
 }
}
```

- If *S* is negative, it is the magnitude of #waiting processes.
- Semaphores ops should be executed atomically. Disabling interrupts in single proc env during wait and signal() op can work.
- But not in Multi proc env. Disabling interrupts is time taking.
- We still do not avoid busy waiting completely but having moved the busy waiting from the entry section to critical section improves efficiency.

- **Deadlocks and starvation**

- Indefinite blocking is starvation

- **Priority Inversion**

- Happens when there are more than two priorities among process for ex. High, Middle, Low priority
- When a higher priority process needs to read or modify kernel data that are with a

lower priority process,

- But if a process with priority **M** preempts **L** then the **H** can be stuck until the process **M** completes execution and sends **signal()** .
- This is **priority inversion**. Solution to this is
  - Keep only two priorities.
  - Or use **priority inheritance** which assigns High priority to an L process required by the H process until the H has received its response from the L process. And then the priorities revert back to their original values.

### ***PRIORITY INVERSION AND THE MARS PATHFINDER***

Priority inversion can be more than a scheduling inconvenience. On systems with tight time constraints—such as real-time systems—priority inversion can cause a process to take longer than it should to accomplish a task. When that happens, other failures can cascade, resulting in system failure.

Consider the Mars Pathfinder, a NASA space probe that landed a robot, the Sojourner rover, on Mars in 1997 to conduct experiments. Shortly after the Sojourner began operating, it started to experience frequent computer resets. Each reset reinitialized all hardware and software, including communications. If the problem had not been solved, the Sojourner would have failed in its mission.

The problem was caused by the fact that one high-priority task, “bc\_dist,” was taking longer than expected to complete its work. This task was being forced to wait for a shared resource that was held by the lower-priority “ASI/MET” task, which in turn was preempted by multiple medium-priority tasks. The “bc\_dist” task would stall waiting for the shared resource, and ultimately the “bc\_sched” task would discover the problem and perform the reset. The Sojourner was suffering from a typical case of priority inversion.

The operating system on the Sojourner was the VxWorks real-time operating system, which had a global variable to enable priority inheritance on all semaphores. After testing, the variable was set on the Sojourner (on Mars!), and the problem was solved.

A full description of the problem, its detection, and its solution was written by the software team lead and is available at [http://research.microsoft.com/en-us/um/people/mbj/mars\\_pathfinder/authoritative\\_account.html](http://research.microsoft.com/en-us/um/people/mbj/mars_pathfinder/authoritative_account.html).

◦

# Chap5\_Classic Problems of Synchronization

## The bounded Buffer Problem

- In our problem, the producer and consumer processes share the following data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```

We assume that the pool consists of  $n$  buffers, each capable of holding one item. The `mutex` semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The `empty` and `full` semaphores count the number of empty and full buffers. The semaphore `empty` is initialized to the value  $n$ ; the semaphore `full` is initialized to the value 0.

## The Readers-Writer problem

- Problems
  - Two readers accessing the shared data, no problems.
  - One writer and some other(Reader/Writer) process access the database simultaneously, then **there is problem**. This problem is used to test synchronization primitive
- Several variations of this problem:
  - First type: No reader should wait for another reader. A situation where a writer process never writes. It is referred to as the "First reader-writer" problem.
  - Second type: a writer performs its write as soon as possible, and no new reader should start reading until then. **Once a writer is ready to write, no "newly arrived reader" is allowed to read**
- Solution to first RW problem
  - ```
semaphore rw_mutex = 1;  
semaphore mutex = 1;  
int read_count = 0;
```
 - `mutex` and `rw_mutex` are initialised to 1; `read_count`--0.
 - `rw_mutex` common to both reader and writer. A mutual exclusion semaphore for writers
 - `read_count` —→ #processes reading the object

- **mutex** —→ for mutual exclusion

```
do {
    wait(rw_mutex);

    . . .
    /* writing is performed */
    . . .
    signal(rw_mutex);
} while (true);
```

Figure 5.11 The structure of a writer process.

- if a writer is in the critical section and n readers are waiting, then one reader is queued on `rw_mutex`, and $n - 1$ readers are queued on `mutex`.
- When a process wishes only to read shared data, it requests the reader–writer lock in read mode.
- A process wishing to modify the shared data must request the lock in write mode. Multiple processes are permitted to concurrently acquire reader–writer lock in read mode, but only one process may acquire the lock for writing, as exclusive access is required for writers.

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    . . .
    /* reading is performed */
    . . .
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Figure 5.12 The structure of a reader process.

- Reader writer's problem is helpful in situation where:
 - It is **easy to identify processes which only read and processes which only write**
 - There are **more reader's than writers**. Because writer locks require **more overhead**. The increased concurrency of allowing multiple readers compensates for the overhead involved in setting up the reader–writer lock.

The Dining's Philosopher Problem



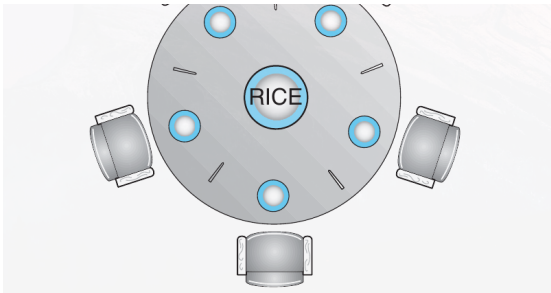


Figure 5.13 The situation of the dining philosophers.

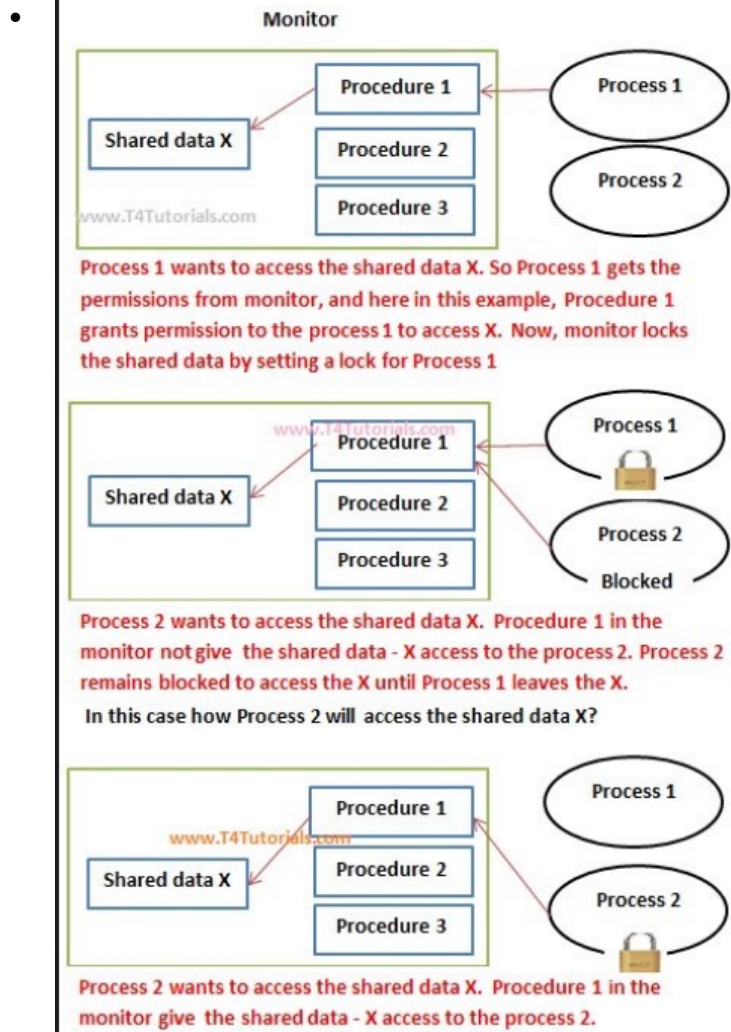
- 5 philosophers, 5 chopsticks, a person needs two to eat their noodles, but would have to wait for until two chopsticks become available to him.
- No preemption when someone else is eating.
- Problem arises when all philosophers get hungry at the same time, and pick up their chopsticks, but they can't eat with just one chopstick.
- `semaphore chopstick[5];`

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    . . .
    /* eat for awhile */
    . . .
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    . . .
    /* think for awhile */
    . . .
} while (true);
```

Figure 5.14 The structure of philosopher i .

- **Solutions:**
 - Allow only 4 philosophers to be sitting simultaneously at the table
 - Allow them to pick up only if both the chopsticks are available
 - Asymmetric solution: odd number pehle left wala uthao, even numbered pehle right wala uthao
- *Note, however, that any satisfactory solution to the dining-philosophers problem must guard against the possibility that one of the philosophers will starve to death. A deadlock-free solution does not necessarily eliminate the possibility of starvation.*

Chaps_monitors



- Semaphores can be very useful for solving concurrency problems, **but only if programmers use them properly**. If even one process fails to abide by the proper use of semaphores, either accidentally or deliberately, then the whole system breaks down. (And since concurrency problems are by definition rare events, the problem code may easily go unnoticed and/or be heinous to debug.)
- For this reason a higher-level language construct has been developed, called **monitors**
- Using semaphores incorrectly can result in deadlock and starvation states.
- Situations such as:
 - Interchanged wait and signal operation
 - ```

signal(mutex);
...
critical section
...
wait(mutex);

```
    - Several processes may be executing in their critical sections simultaneously, violating the mutual-exclusion requirement.

- This error may be discovered only if several processes are simultaneously active in their critical sections

## 2. Signal interchanges with wait operation

- ```
wait(mutex);  
...  
critical section  
...  
wait(mutex);
```

3. Process omits the wait(mutex), or the signal(mutex), or both. In this case, either mutual exclusion is violated or a deadlock will occur.

- To deal with such errors, researchers have developed high-level language constructs. In this section, we describe one fundamental high-level synchronization construct—the monitor type

- ```
monitor monitor name
{
 /* shared variable declarations */

 function P1 (. . .) {
 . . .
 }

 function P2 (. . .) {
 . . .
 }

 .
 .
 .
 function Pn (. . .) {
 . . .
 }

 initialization_code (. . .) {
 . . .
 }
}
```

**Figure 5.15** Syntax of a monitor.

- A monitor is essentially a class, in which all data is private, and with the special restriction that **only one method within any given monitor object may be active at the same time**. An additional restriction is that monitor methods may only access the shared data within the monitor and any data passed to them as parameters. I.e. they cannot access any data external to the monitor

data passed to them as parameters. i.e. they cannot access any data external to the monitor.

- Monitors provide mutual exclusion by providing a set of condition variables
- In order to fully realize the potential of monitors, we need to introduce one additional new data type, known as a **condition**.
  - A variable of type condition has only two legal operations, **wait** and **signal**. I.e. if X was defined as type condition, then legal operations would be X.wait( ) and X.signal( )
  - The wait operation blocks a process until some other process calls signal, and adds the blocked process onto a list associated with that condition.
  - The signal process does nothing if there are no processes waiting on that condition. Otherwise it wakes up exactly one process from the condition's list of waiting processes. (Contrast this with counting semaphores, which always affect the semaphore on a signal call).

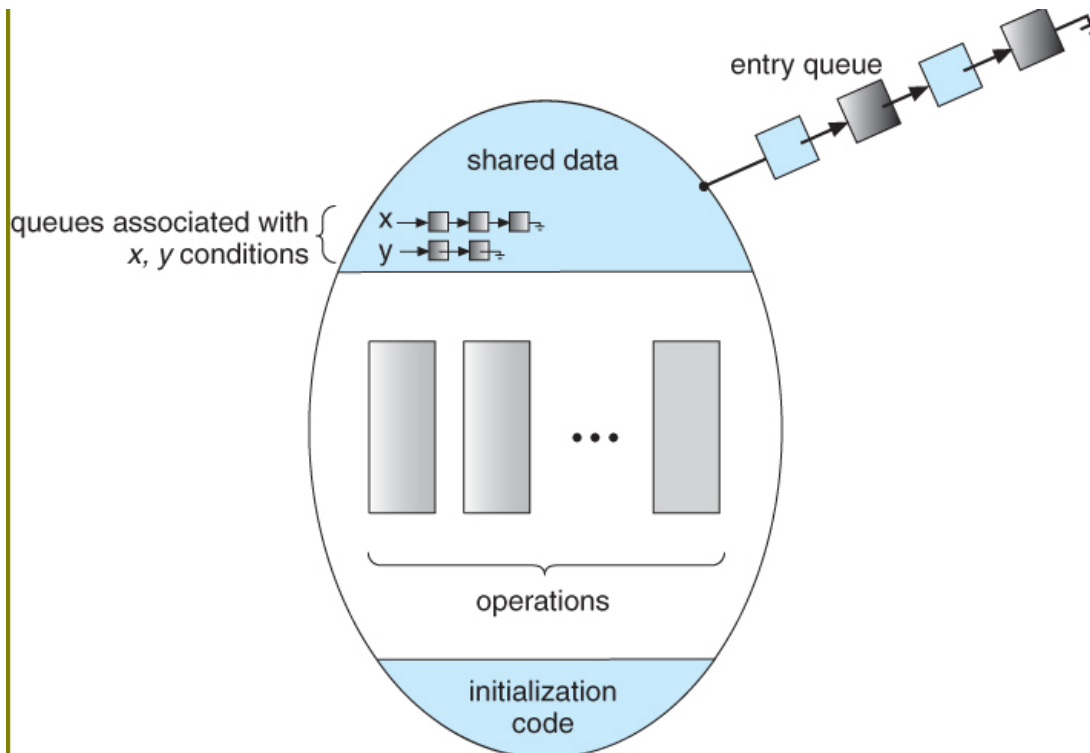


Figure 5.17 - Monitor with condition variables

Can be used to solve the dining philosophers problem,

## • Problem

- If process P within the monitor issues a signal that would wake up process Q also within the monitor, then there would be two processes running simultaneously within the monitor, violating the exclusion requirement. Accordingly there are two possible solutions to this dilemma:
  - **Signal and wait** - When process P issues the signal to wake up process Q, P then waits, either for Q to leave the monitor or on some other condition.
  - **Signal and continue** - When P issues the signal, Q waits, either for P to exit the monitor or for some other condition.
- There are arguments for and against either choice. **Concurrent Pascal offers a third alternative** - The signal call causes the signaling process to immediately exit the monitor,

so that the waiting process can then wake up and proceed.

- **Dining's Philosophers problem ka solution (Using monitors)**

- This solution to the dining philosophers uses monitors, and the restriction that a philosopher may only pick up chopsticks when both are available. There are also two key data structures in use in this solution:
  1. `enum { THINKING, HUNGRY, EATING } state[ 5 ]`; A philosopher may only set their state to eating when neither of their adjacent neighbors is eating. ( `state[ ( i + 1 ) % 5 ] != EATING && state[ ( i + 4 ) % 5 ] != EATING` ).
  2. `condition self[ 5 ]`; This condition is used to delay a hungry philosopher who is unable to acquire chopsticks.
- In the following solution philosophers share a monitor, `DiningPhilosophers`, and eat using the following sequence of operations:
  1. `DiningPhilosophers.pickup( )` - Acquires chopsticks, which may block the process.
  2. eat
  3. `DiningPhilosophers.putdown( )` - Releases the chopsticks.

```
monitor DiningPhilosophers
{
 enum {THINKING, HUNGRY, EATING} state[5];
 condition self[5];

 void pickup(int i) {
 state[i] = HUNGRY;
 test(i);
 if (state[i] != EATING)
 self[i].wait();
 }

 void putdown(int i) {
 state[i] = THINKING;
 test((i + 4) % 5);
 test((i + 1) % 5);
 }

 void test(int i) {
 if ((state[(i + 4) % 5] != EATING) &&
 (state[i] == HUNGRY) &&
 (state[(i + 1) % 5] != EATING)) {
 state[i] = EATING;
 self[i].signal();
 }
 }

 initialization_code() {
 for (int i = 0; i < 5; i++)
 state[i] = THINKING;
 }
}
```

**Figure 5.18** A monitor solution to the dining-philosopher problem.

- Although this solution can still lead to starvation.
  - If process  $i$  completes eating and its neighbor starts eating, while process  $i$  again becomes hungry, can lead to starvation. So we can have a priority queue for the philosophers with first come first serve as an example.

## Implementing a Monitor using Semaphore

- For each monitor, a semaphore `mutex` (initialized to 1) is provided, to control access to monitor.
- A process must execute `wait(mutex)` before entering the monitor and must execute `signal(mutex)` after leaving the monitor.
- Since a signaling process must wait until the resumed process either leaves or waits, an additional semaphore, `next`, is introduced, initialized to 0.
- The signaling processes can use `next` to suspend themselves, after they are already "inside" the monitor.
- An integer variable `next_count` is also provided to count the number of processes suspended on `next`. Thus, each external function  $F$  is replaced by

```
wait(mutex);
...
body of F
...
if (next_count > 0)
 signal(next);
else
 signal(mutex);
```

- The operation `x.wait()` can now be implemented as

```
x_count++;
if (next_count > 0)
 signal(next);
else
 signal(mutex);
wait(x_sem);
x_count--;
```

- The operation `x.signal()` can be implemented as

```
if (x_count > 0) {
 next_count++;
 signal(x_sem);
 wait(next);
 next_count--;
}
```

- The above operation does nothing when `x_count==0`, meaning when no operations are

available, which is the difference between **signal in mutex** and **signal in monitors**.

## Resuming Processes within a Monitor

- Multiple Processes waiting on a single process, so **which one to wake up on signal call**
  - FCFS approach
  - Integral Priorities

```
monitor ResourceAllocator
{
 boolean busy;
 condition x;

 void acquire(int time) {
 if (busy)
 x.wait(time);
 busy = true;
 }

 void release() {
 busy = false;
 x.signal();
 }

 initialization_code() {
 busy = false;
 }
}
```

**Figure 5.19** A monitor to allocate a single resource.

- Unfortunately the use of monitors to restrict access to resources **still only works if programmers make the requisite acquire and release calls properly.**
- **One option would be to place the resource allocation code into the monitor, thereby eliminating the option for programmers to bypass or ignore the monitor, but then that would substitute the monitor's scheduling algorithms for whatever other scheduling algorithms may have been chosen for that particular resource.**
- Unfortunately, the monitor concept cannot guarantee that the preceding access sequence will be observed. In particular, the following problems can occur:
  - • **A process might access a resource without first gaining access permission to the resource.**
  - • **A process might never release a resource once it has been granted access to the resource.**
  - • **A process might attempt to release a resource that it never requested.**
  - • **A process might request the same resource twice (without first releasing the resource).**
- We **must check two conditions to establish the correctness** of this system.
  - **First, user processes must always make their calls in the monitor in a correct sequence.**
  - **Second, we must be sure that an uncooperative process does not simply ignore the mutual-**

exclusion gateway provided by the monitor and try to access the shared resource directly, without using the access protocols.

- Only if these two conditions can be ensured can we guarantee that no time-dependent errors will occur and that the scheduling algorithm will not be defeated.

#### *JAVA MONITORS*

Java provides a monitor-like concurrency mechanism for thread synchronization. Every object in Java has associated with it a single lock. When a method is declared to be synchronized, calling the method requires owning the lock for the object. We declare a synchronized method by placing the synchronized keyword in the method definition. The following defines `safeMethod()` as synchronized, for example:

```
public class SimpleClass {
 . . .
 public synchronized void safeMethod() {
 . . .
 /* Implementation of safeMethod() */
 . . .
 }
}
```

Next, we create an object instance of `SimpleClass`, such as the following:

```
SimpleClass sc = new SimpleClass();
```

Invoking `sc.safeMethod()` method requires owning the lock on the object